

Simulador de Escalonamento de Processos

Aluno: Samuel

Curso: Tecnologias em Sistemas de Computação

Disciplina: Sistemas Operacionais

Data de entrega: 02/06/2025

Algoritmos implementados:

- FIFO (First-In, First-Out)**
- SJF (Shortest Job First)**

Código fonte do simulador:

```
class Processo:
    def __init__(self, size, id, ordem):
        self.id = id
        self.ordem = ordem
        self.size = size

    def __repr__(self):
        return f'ID = {self.id:02}, Size = {self.size:02}, Ordem = {self.ordem}'

class Processado:
    def __init__(self, processo, turn, tw, ordem):
        self.ordem = ordem
        self.processo = processo
        self.turnAround = turn
        self.timeWait = tw

class Processador:
    def __init__(self):
        self.processos = []

    def addProcess(self, process):
        self.processos.append(process)

    def showProcess(self):
        print('Lista de processos:')
        for i in self.processos:
            print(i)
```

```

print()

def fifo(self):
    timeWaits = 0
    result = []
    sorted = []

    for i in range(1, len(self.processos) + 1):
        for o in self.processos:
            if o.ordem == i:
                sorted.append(o)

    for i, processo in enumerate(sorted):
        if i == 0:
            result.append(Processado(processo, processo.size, 0, processo.ordem))
        else:
            result.append(Processado(processo, processo.size + timeWaits, timeWaits, processo.ordem))

        timeWaits += processo.size
    total_ta = 0
    total_tw = 0

    print('-----')
    print('          |FIFO|')
    print('ID      SIZE      TA      TW      OD')
    for i in result:
        print(f'{i.processo.id:02}      {i.processo.size:02}      {i.turnAround:02}      {i.timeWait:02}')
        total_ta += i.turnAround
        total_tw += i.timeWait

    print()
    print(f'Tempo médio de turnAround: {total_ta / len(self.processos):.2f}')
    print(f'Tempo médio de timeWaits: {total_tw / len(self.processos):.2f}')

def sjf(self):
    timeWaits = 0
    result = []
    sortedList = sorted(self.processos, key=lambda p: p.size)

    for i, processo in enumerate(sortedList):
        if i == 0:
            result.append(Processado(processo, processo.size, 0, processo.ordem))
        else:
            result.append(Processado(processo, processo.size + timeWaits, timeWaits, processo.ordem))
        timeWaits += processo.size

    total_ta = 0
    total_tw = 0

    print('-----')
    print('          |SJF|')
    print('ID      SIZE      TA      TW      OD')
    for i in result:
        print(f'{i.processo.id:02}      {i.processo.size:02}      {i.turnAround:02}      {i.timeWait:02}')

```

```

        total_ta += i.turnAround
        total_tw += i.timeWait
    print()
    print(f'Tempo médio de turnAround: {total_ta / len(sortedList):.2f}')
    print(f'Tempo médio de timeWaits: {total_tw / len(sortedList):.2f}')

class Execucao:
    @staticmethod
    def inicio(core):
        while True:
            try:
                qtd = int(input('\nQuantos processos você gostaria de adicionar?: '))
                break
            except ValueError:
                print("Por favor, insira um número inteiro válido.")

        for i in range(1, qtd + 1):
            while True:
                try:
                    size, ordem = map(int, input(f'Insira o tamanho (int) e ordem (int) (separados por espaço): ').split())
                    break
                except ValueError:
                    print("Tamanho inválido. Digite um número inteiro.")
            core.addProcess(Processo(size, i, ordem))

        print('\n-----')
        core.showProcess()
        print('-----\n')

    @staticmethod
    def metodo(core):
        core.fifo()
        core.sjf()

        while True:
            continuar = input('Gostaria de reiniciar a simulação (R) ou fechar (F)? ').upper()
            if continuar in ['R', 'F']:
                return continuar
            print("Opção inválida. Digite R ou F.")

def main():
    core = Processador()
    while True:
        Execucao.inicio(core)
        resposta = Execucao.metodo(core)

        if resposta == 'R':
            core = Processador()
        elif resposta == 'F':
            print('Simulação finalizada.')
            break

if __name__ == "__main__":
    main()

```

